



*Software Requirements Specification
Version 1.0*

FreshGuard

Project Name: FreshGuard

Theme: Zero Food Waste

Category: Smart IoT Revolution

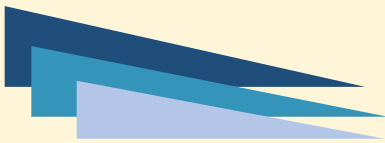
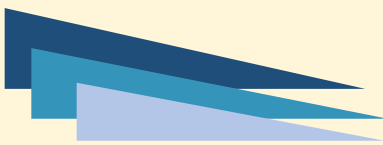


Table of Contents

1.1	Background and Necessity of the System	2
1.2	Proposed Solution.....	3
1.3	Purpose of the Document.....	7
1.4	Scope of Project.....	7
1.5	Constraints	8
1.6	Functional Requirements.....	8
1.7	Non-Functional Requirements	13
1.8	Interface Requirements.....	15
1.8.1	Hardware.....	15
1.8.2	Software.....	17
1.9	Project Deliverables.....	18



1.1 Background and Necessity of the System

Food waste is a major environmental, social, and economic concern across the world. A considerable quantity of food purchased for households is discarded because it expires, becomes spoiled or is forgotten inside refrigerators and kitchen storage containers. Improper food storage, unsuitable temperature, excessive humidity, prolonged storage duration and a lack of expiry reminders are some of the common causes of household food waste. Consumers may also find it difficult to determine whether stored food is still fresh, should be consumed soon or has become unsafe.

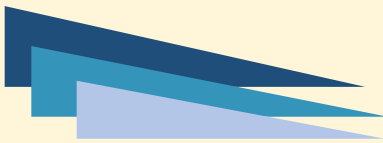
Food spoilage may not always be visible immediately. Changes in gas concentration or air-quality conditions within the storage environment may occur as food deteriorates. Unsuitable temperature and humidity conditions can accelerate food deterioration and microbial growth and reduce food quality.

The growing use of Internet of Things (IoT) technology provides an opportunity to develop intelligent food-storage systems. Sensors can continuously monitor temperature, humidity, storage duration and changes in gas concentration or air-quality conditions within the storage environment. The collected information can be processed and displayed through a mobile or Web-based dashboard.

The inclusion of IoT technology can significantly improve food management at home or restaurants. Real-time sensor data, automated reminders and freshness-status indicators can help users make informed decisions before food becomes unsuitable for consumption.

FreshGuard is proposed as a prototype for household or small-restaurant food-storage monitoring. The system is intended to provide freshness-condition indicators and storage alerts to users or authorized restaurant staff. It provides these based on sensor readings, storage duration, expiry information and predefined rules.





1.2 Proposed Solution

The proposed system, **FreshGuard**, will use IoT technology to monitor food-storage conditions in refrigerators, kitchen storage areas, food containers, and small-restaurant storage areas. Each food container should have a QR-code sticker containing or linking details such as the food name, date of manufacture, expiry date, and a unique identification ID. Scanning the QR code should allow the food item to be identified and its details to be retrieved or updated in the system. The unique ID should help **FreshGuard** track individual food items and determine their freshness condition based on the stored details and monitored sensor readings.

Temperature, humidity, gas and door sensors will send real-time data to a selected controller such as a Raspberry Pi, NodeMCU or ESP32. Teams should select and implement one suitable controller of architecture. The system will also track the food category, storage duration and expiry/best-before date where available, or a configured storage-duration limit.

The processing unit will compare the collected data with predefined threshold values and assign one of the following freshness-condition statuses:

- **Fresh/Normal:** Storage conditions are within the configured acceptable range.
- **Use Soon:** The food item is approaching its configured storage-duration or expiry limit.
- **Check Food:** Requires inspection because abnormal conditions or configured limits have been detected.

If abnormal temperature, humidity or gas sensor readings are detected, FreshGuard should activate the local buzzer/alarm and display a warning on the LCD. The alert should also be displayed on the dashboard. The system should send a proactive remote notification through the configured approved service.

Processed data should be transferred using the network connectivity supported by the selected controller, such as Wi-Fi or Ethernet. Where remote or cloud storage is used, the records should be sent to the configured Web server, datastore or IoT/cloud platform. Stored records may include food-item details, sensor readings, expiry/best-before information and alerts.

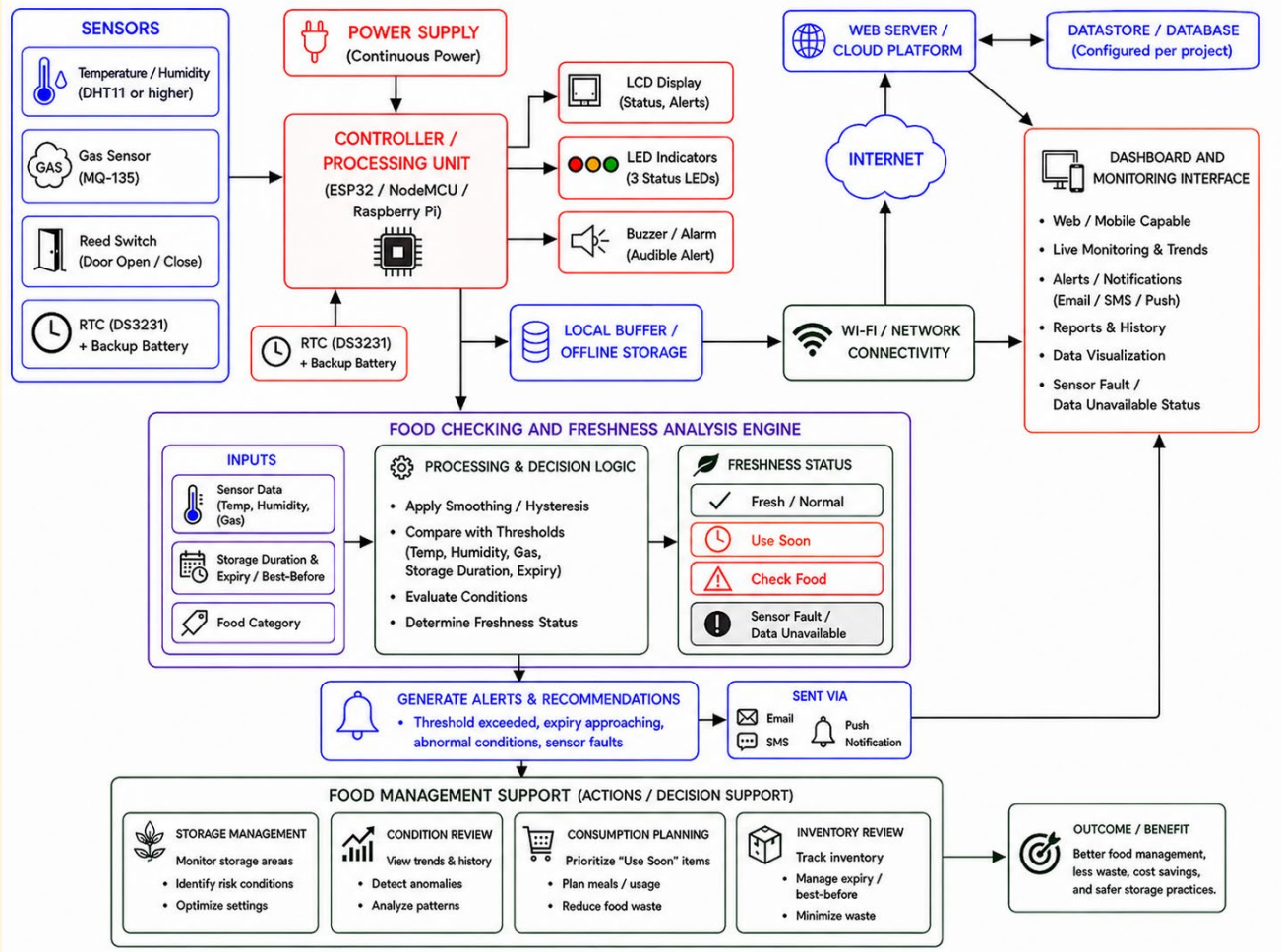
The dashboard will help users or authorized restaurant staff monitor food storage conditions, review inventory, and identify food items that should be used first.



Note: If a refrigerator is not available due to space, availability, or other practical constraints, teams may use a small, insulated box, cooler box, storage container, or similar enclosed setup to simulate refrigerated food storage. The selected setup should allow the team to demonstrate the required sensing, monitoring, QR-based identification, freshness assessment and alert functions of the FreshGuard system.

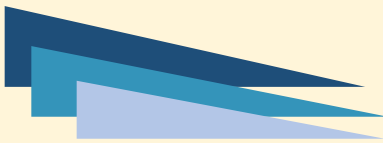


The sample architecture for the FreshGuard IoT-based Smart Food Freshness Monitor is as follows:



Sample Architecture of the System

The diagram represents the workflow of the IoT-based Smart Food Freshness Monitor System. The DHT11 (or higher) sensor monitors temperature and humidity within the storage environment, while the MQ-135 monitors changes in gas concentration or air-quality readings that may indicate abnormal storage conditions. The reed switch checks whether the refrigerator or storage door is open or closed. These sensor inputs are sent to the selected Raspberry Pi, NodeMCU or ESP32 controller, which acts as the main processing unit. If a Raspberry Pi is selected and analog MQ-135 readings are required, an ADC module such as MCP3008 or ADS1115 should be used.



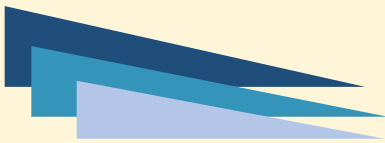
The controller processes the incoming data and supports the Food Processing and Freshness Analysis module. The system combines sensor readings, storage-duration information, expiry-date information and food-category data. The Freshness Decision Engine compares these values with predefined thresholds and assigns one of the following freshness-condition statuses: Fresh/Normal, Use Soon or Check Food. Fresh/Normal indicates that monitored storage conditions are within the configured acceptable range. Use Soon indicates that the item is approaching its configured storage-duration or expiry limit. Check Food indicates that abnormal conditions have been detected and human inspection is required.

Environmental sensors may monitor a shared storage zone or individual food container depending on the prototype design. Wherever a sensor is shared across a storage zone, individual food-item status should combine item-specific storage and expiry information with the environmental readings for that zone. A shared gas sensor should not be used to identify which specific food item caused a change in gas readings.

The controller processes the sensor inputs and supports the Freshness Analysis Engine. Required sensor records, alerts and synchronization data are transferred through the network connection supported by the selected controller, such as Wi-Fi or Ethernet. The Web server or IoT/cloud platform exchanges information with the configured datastore and provides the required data to the dashboard. The dashboard displays sensor readings, freshness-condition status, alerts, expiry/best-before information, reports and data visualizations. The controller directly operates the local buzzer/alarm, LED indicators and LCD. Alerts generated by the Freshness Analysis Engine are provided locally and are also delivered to the dashboard through the configured network/cloud path.

The alert and recommendation output helps support food-management activities such as storage management, condition review, consumption planning, and inventory review. For example, the system may show:

- **Fresh/Normal:** Milk is stored under currently acceptable monitored conditions.
- **Use Soon:** Tomatoes should be consumed soon as the expiry date is approaching.
- **Check Food:** Check the stored cheese. Its storage duration has been exceeded, and abnormal gas readings have been detected.



1.3 Purpose of the Document

The purpose of this document is to provide a detailed description of the IoT-based food freshness monitoring system titled **FreshGuard**.

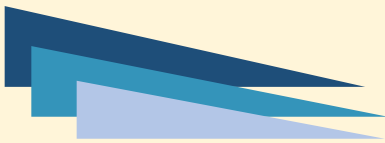
This document is intended for developers, evaluators, mentors and other stakeholders involved in the development and evaluation of the project.

1.4 Scope of Project

The scope of the project is to develop an IoT-based prototype for household or small-restaurant food-storage monitoring that helps users reduce avoidable food waste.

FreshGuard will monitor food stored inside refrigerators, food containers, kitchen storage areas and small-restaurant storage areas. The system will register food items, track storage duration, monitor environmental conditions and provide timely alerts. FreshGuard does not determine food safety or certify whether food is safe for consumption. Users should follow applicable storage guidance, product labels and appropriate food-safety practices beyond this system when deciding whether food should be consumed or discarded.





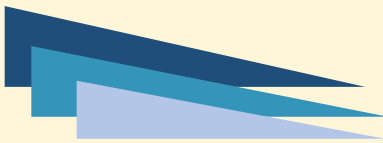
1.5 Constraints

The system's accuracy depends on sensor calibration, placement, food type, and proper food handling. Continuous power is required for active monitoring. Internet connectivity is required only for cloud synchronization and remote notifications; local sensing and alerts should continue during temporary network unavailability. Different foods require different freshness thresholds, and large-scale installation may increase cost.

1.6 Functional Requirements

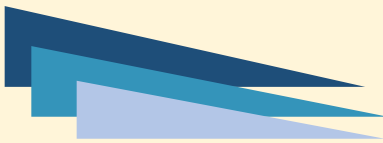
Following are the functional requirements of the system:

- i. **Data Collection** – The system should collect real-time data from temperature, humidity, gas, and door sensors. It should also retrieve and store container-level details through the QR-code sticker attached to each food container, including the food name, category, date of manufacture, storage date, expiry or best-before date where available, or a configured storage-duration limit, along with a unique identification ID for tracking and freshness evaluation.
- ii. **Temperature and Humidity Sensor** – The DHT11 (or higher) sensor should measure temperature and humidity within the monitored storage area and send the readings to the selected controller.
- iii. **Safe Sensor Interfacing** – The selected controller and sensors should use appropriate supply voltages, common grounding and safe signal levels. Suitable signal-conditioning, voltage-divider or level-shifting components should be used where required. Unsafe voltage levels should not be connected directly to ESP32 or NodeMCU input pins.
- iv. **Gas-Level Monitoring** – The MQ-135 should monitor relative gas/air-quality readings that may indicate abnormal storage conditions. Gas readings alone should not be used to confirm food spoilage. Abnormal readings beyond configured thresholds should generate an alert.
- v. **Sensor Calibration and Baseline** – Before freshness monitoring begins, teams should calibrate sensors were supported or establish and



document baseline readings under controlled conditions. The MQ-135 should be allowed to warm up and stabilize according to the selected sensor/module requirements before baseline readings are recorded. Teams should not report calibrated gas concentration or ppm values unless suitable calibration has been performed. Normal baseline values and configured abnormal thresholds should be documented. For DHT11 (or higher) and MQ-135 readings, the system should use suitable averaging/smoothing, hysteresis or consecutive-reading validation to reduce false alerts. Debounce logic should be applied to the reed-switch input where required.

- vi. Door Monitoring** – A reed switch should detect whether the refrigerator or storage door is open. If it remains open for too long, the system will activate a buzzer and send a notification.
- vii. Sensor Fault Handling** – If a required sensor returns invalid or missing readings, FreshGuard should identify the fault where technically possible. It should not display a misleading Fresh or Normal status when a sensor or input fault is detected. The dashboard or LCD should display a separate ‘Sensor Fault/Data Unavailable’ state for the affected sensor or monitored condition. The faults will should be recorded and the user will should be notified where appropriate. The system will should continue safe operation using the remaining valid inputs where possible.
- viii. Food Registration** – The system should allow users to register food items with details such as name, category, quantity, storage date, storage location and expiry/best-before date where available, it will also configure recommended storage-duration limit.
- ix. Storage and Expiry Tracking** – The DS3231 RTC, with its backup battery installed, should retain date and time during temporary loss of controller power. Active sensing and processing require power to the controller. The application should use the RTC or system time to maintain item timestamps and calculate storage duration.
- x. Food Freshness Analysis** – The system should analyze:
 - Temperature and humidity
 - Gas sensor reading or relative gas/air-quality reading
 - Storage duration



- Expiry/best-before date, where available, or configured storage-duration limit
- Food category

Teams should prepare and document a Food Threshold Matrix containing the food category, acceptable temperature range, humidity range, and storage-duration limit. The matrix should also include the expiry condition, gas-sensor baseline, configured abnormal threshold, and resulting freshness-condition status or alert. Threshold values should be based on cited food-storage guidance or manufacturer/source information where available. Wherever authoritative values are not available for the prototype, teams should clearly identify the selected values as controlled prototype/demo assumptions and should not present them as certified food-safety standards. Gas-sensor baseline and configured abnormal threshold values may be defined at the storage-zone or container level. Where one MQ-135 sensor monitors a shared storage zone, the gas reading should not be attributed to any specific food item.

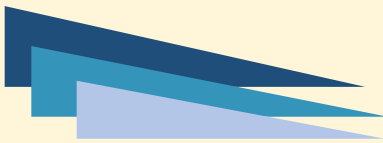
xi. Freshness Status – The system should assign one of the following freshness-condition statuses based on the monitored conditions and configured rules:

- **Fresh/Normal:** Storage conditions are within the configured acceptable range.
- **Use Soon:** The food item is approaching its configured storage-duration or expiry limit.
- **Check Food:** Requires inspection because abnormal conditions or configured limits have been detected.

Teams should prepare a Freshness Decision Rule Matrix defining how combinations of temperature, humidity, gas, storage-duration and expiry conditions result in Fresh/Normal, Use Soon or Check Food status.

xii. Threshold Alerts – The administrator should be able to configure limits for temperature, humidity, gas sensor readings, storage duration and door-open time. When a confirmed or persistent threshold violation is detected, the system should:

- Activate the buzzer
- Display a warning on the LCD
- Store the alert

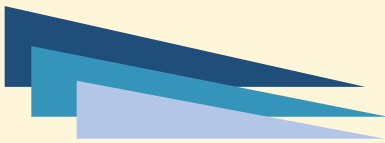


- Display the alert on the dashboard
- Send a mandatory proactive remote notification through e-mail, Blynk/app push notification or another approved remote-notification service

- xiii. Data Processing and Storage** – The selected controller should process sensor readings and send the results to the configured datastore or IoT/cloud platform, where applicable.
- xiv. Wi-Fi and Cloud Connection** – The system should use network connectivity supported by selected controller, such as Wi-Fi or Ethernet, to transfer sensor readings, food records and alerts to the Web server or cloud platform.
- xv. Offline Operation and Synchronization** – During temporary network unavailability, local sensor monitoring, freshness analysis and local alerts should continue. Important unsent alerts and events should be buffered locally using storage available on the selected controller. High-frequency sensor readings may also be buffered where local storage capacity permits. Each buffered event should contain a unique event ID, timestamp and synchronization status. When connectivity is restored, unsynchronized records should be transferred to the configured datastore or IoT/cloud platform without creating duplicate records.
- xvi. Data Display** – The LCD should display temperature, humidity, gas sensor reading, freshness-condition status and warning messages. If a critical sensor fault is detected, the LCD or dashboard should display ‘Sensor Fault/Data Unavailable’ instead of a misleading Fresh/Normal indication.

LED indicators should show the following status indications:

- Green – Fresh/Normal
- Yellow – Use Soon
- Red – Check Food



xvii. Dashboard and Monitoring Interface – The dashboard should display:

- Sensor readings
- Food inventory
- Expiry/best-before information or configured storage-duration limits
- Storage duration
- Freshness status
- Sensor-fault/data-unavailable status
- Active alerts

One responsive Web dashboard or an approved IoT-platform dashboard is sufficient for the project.

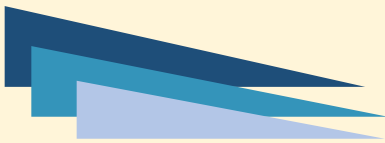
xviii. Alerts and Recommendations – The system should send alerts and recommendations such as:

- Use the food soon
- Check the storage temperature
- Close the refrigerator door
- Inspect the food before use
- If human inspection determines that the food is unsuitable for use, follow the appropriate disposal procedure.

xix. Food Management Support – The dashboard should support inventory review, food-condition checks and consumption planning for household or small-restaurant use.

xx. Data Visualization – The system should represent readings and food records through charts, tables and dashboard summaries using the selected dashboard or IoT/cloud platform. The selected IoT platform may serve as the dashboard and data-visualization interface; a separate second dashboard is not mandatory.

xxi. IoT Testing – Teams should prepare and execute an IoT Test Matrix. The matrix should contain the following columns: Test ID, Test Scenario, Input/Condition, Expected Result, Actual Result, Pass/Fail Status and Evidence.



At minimum, the IoT Test Matrix should cover:

- Normal temperature/humidity/gas sensor readings
- High temperature
- Abnormal humidity
- Abnormal gas reading
- Door left open beyond the configured timeout
- Use Soon condition
- Check Food condition
- Invalid/disconnected sensor and ‘Sensor Fault/Data Unavailable’ status verification
- Temporary network failure
- Network reconnection, buffered-event synchronization and duplicate-prevention verification
- Local alert verification
- Remote notification verification
- Freshness Decision Rule verification

In addition to the mandatory requirements, teams may implement the following optional functionalities:

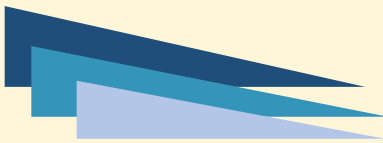
- The system may include recipe suggestions, QR or RFID tracking, purchase-list generation, food-waste reports and AI-based spoilage prediction.

1.7 Non-Functional Requirements

Following non-functional requirements define the expected quality and operational characteristics of the system:

Portability: The prototype components should be suitable for placement in or near the selected monitored food-storage environment, subject to environmental and electrical-safety requirements.

Accuracy: Sensor readings should remain within the manufacturer-stated operating tolerance under the prototype test conditions. Sensor baseline/calibration results should be documented before testing. DHT11 (or higher) and MQ-135 are prototype-grade sensors; readings may vary according to manufacturer tolerances, sensor placement and environmental conditions.



Information Clarity: Freshness-condition status, alerts, charts and reports should be clear, consistent and easy to understand.

Space Efficiency: The system should monitor different food-storage areas without occupying excessive space.

Performance Efficiency: Under normal operating conditions, the system should generate a local alert within 2–5 seconds of a confirmed threshold violation. The dashboard should display the updated condition within the configured refresh interval under normal network conditions.

Scalability: The system should support additional sensors, food items and monitored storage locations without major redesign.

Security: User credentials, food records, sensor data and alerts should be stored and transmitted securely.

Reliability: The system should provide continuous monitoring with minimal interruption.

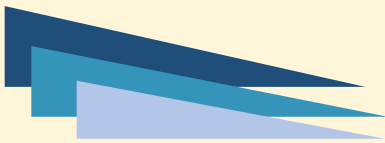
Environmental and Electrical Safety: Sensors and electronic components used inside or near refrigerators, food-storage areas or containers should be positioned safely and protected from condensation, excessive moisture and direct food contact where applicable. The prototype should use safe low-voltage power supplies suitable for the selected controller and sensors.



These requirements represent the minimum mandatory expectations for the project. All functional and non-functional requirements specified in this SRS must be implemented.

Once the mandatory requirements are complete, teams may implement additional relevant features.

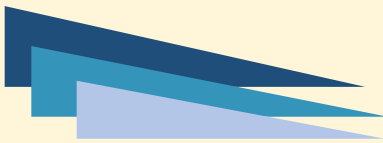
Do NOT copy content or code from GPTs or other AI tools, although you are permitted to use images generated by AI tools for any visual representation purposes. It is mandatory to mention such tools used in case you add any AI-generated images.



1.8 Interface Requirements

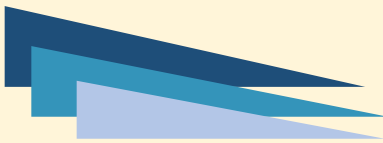
1.8.1 Hardware

Components	Quantity
Raspberry Pi/NodeMCU/ESP32; select one supported controller.	1
Buzzer/Alarm	1
Reed Switch with Magnet	1
Suitable regulated power supply/USB adapter, compatible with the selected controller and sensors	1
Temperature and Humidity Sensor: DHT11 (or higher)	1
Gas Sensor: MQ-135	1
DS3231 Real-Time Clock Module	1
RTC backup battery compatible with the selected DS3231 module	1



Components	Quantity
ADC Module: MCP3008 or ADS1115, if a Raspberry Pi is used with analog MQ-135 measurement	1
Resistors/voltage divider/level-shifting components	As required for safe sensor interfacing
RFID reader and RFID tags	As required (Optional)
QR Code Labels	As required (Optional)
LCD	1
LED indicators	3
Mobile/Laptop	1
Refrigerator and Storage Containers	1
Food Containers	Three or more, as required
Breadboard	1
Jumper Wires	One or more sets, as required

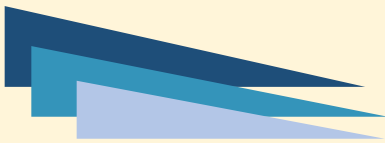
Connectivity Requirement: Network connectivity appropriate to the selected controller should be available where cloud synchronization, dashboard access or remote notification are demonstrated. This may include Wi-Fi or Ethernet, depending on the selected architecture.



1.8.2 Software

Tools and technologies to be used, as applicable:

1. **Frontend:** HTML5/CSS3/JavaScript, Streamlit or another approved interface, where a custom dashboard is implemented. A separate custom frontend is not required when the selected IoT/cloud platform provides the required dashboard
2. **Backend:** Flask or another approved lightweight framework, where a custom backend/Web dashboard is implemented
3. **Programming Language:** Raspberry Pi – Python 3.x; ESP32/NodeMCU – MicroPython where Python is used, or Arduino C/C++ where Arduino IDE is selected
4. **IDE/Development Environment:** Arduino IDE, Visual Studio Code or another approved environment
5. **IoT/Cloud Platform:** ThingSpeak, Blynk IoT or another approved IoT/cloud platform as required
6. **Database:** Use one approved datastore such as JSON, SQLite or Firebase Realtime Database, or use the data-storage capability provided by the selected approved IoT/cloud platform
7. **Operating System:** Raspberry Pi OS, where Raspberry Pi is selected
8. **Libraries/Utilities:** Appropriate ESP32/ESP8266 board package, where Arduino IDE is used; QR-code tools where the optional QR feature is implemented; Fritzing; and other required libraries/utilities



1.9 Project Deliverables

The final project submission should include the following deliverables in the form of a project report:

- Problem Definition
- Design Specifications
- Diagrams such as flowcharts for various Activities, Data Flow Diagrams and so on
- Test Data used in the Project
- Circuit/Wiring Diagram
- Pin-Mapping Table
- Controller and Hardware Specifications
- Sensor Calibration/Baseline Table
- Food Threshold Matrix
- Freshness Decision Rule Matrix
- IoT Test Matrix with Evidence
- Dashboard Screenshots
- Installation Instructions
- Execution Instructions
- README.md

The consolidated project will be submitted as a ZIP file with a README.md file. The README.md will list assumptions, pin configurations for the selected controller, sensors, other components used, and relevant technical specifications. The zip file will also include source code, sensor calibration or baseline information and threshold settings. The Food Threshold Matrix and Freshness Decision Rule Matrix should also be included. In addition, it will contain the IoT Test Matrix with test results and the documented power and wiring plan. Documentation is a very important part of the project; therefore, all crucial aspects must be documented properly. **Documentation should not contain source code.**

Demonstration Video: Submit a video (.mp4 file) demonstrating the working of the FreshGuard prototype, including sensor monitoring, freshness-condition status, threshold alerts, dashboard/data visualization and all other mandatory project functionalities. This is mandatory.

~~~ End of Document ~~~